CS260 - Written Assignment 4

Professor Galen Long, Professsor Kurt Schmidt, Professor Yusuf Osmanlioglu

Summer 2024-25

Total points: 100

- **For any algorithm design questions in this assignment and the course as a whole, give your algorithms in C/Python-like pseudocode *and* describe your algorithm briefly at a high level in plain English.**
- Your pseudocode can omit low-level details like memory allocations and type declarations.
- If you're not sure whether or not to include a detail in your pseudocode, err on the side of including it.
- Small mistakes like off-by-one errors will receive at most a small penalty. We know it's hard to write perfectly correct pseudocode because you can't execute it easily (although running your pseudocode on an example on paper is a great way to debug it).
- Whenever we ask you to design an algorithm, we expect your algorithm to be efficient (within reason). If an asymptotically more efficient algorithm exists, you may lose points if your algorithm is correct but asymptotically slower.
- However, you should prefer a correct but slow algorithm over an incorrect attempt at a more efficient algorithm.

1. (15 points) Solve the following recurrence using the substitution/equation expansion method, then give a tight bound on the recurrence.

$$T(n) = \begin{cases} c & n \leq 0 \\ T(n-3) + 5n & n > 0 \end{cases}$$

   For this problem, you may assume that sums very similar to the arithmetic series are also $O(n^2)$. The arithmetic series states that $1+2+3+4+5+...+n = O(n^2)$. Similar sums like $4+8+12+16+20+...+n$ or $7*1+7*2+7*3+7*4+7*5+...+7*n$ also sum to $O(n^2)$.

   Hint: If you're getting lost in the weeds of the algebra, zoom out and think at a high level. How many substitutions/recursions will it take to reach the base case (logarithmic, linear, quadratic, etc.)? And how much work is done each substitution/recursion? You still need to solve the recurrence using the substitution method, but understanding what the final bound should be will help guide you.

2. Consider a directed graph $G = (V, E)$ and its set of strongly connected components S. For each of the following questions, give the answer and explain a specific situation in which the answer would occur.

   For example, if the question asked, "What's the maximum height of a BST with n nodes?", you could say "$n-1$, which will occur when the elements are inserted in sorted or reverse sorted order".

   (a) (8 points) **Minimum** possible size of S if there are V nodes

   (b) (8 points) **Maximum** possible size of S if there are V nodes

3. Consider a directed graph $G = (V, E)$ that has k strongly connected components. Say we add one edge to G.

(a) (7 points) Is it possible that after adding one edge to G, k won't change?

If yes, give a specific example of a graph and an edge where adding that edge wouldn't change k. If no, explain why.

(b) (7 points) Is it possible that after adding one edge to G, k decreases by $V - 1$?

If yes, give a specific example of a graph and an edge where adding that edge would decrease k by $V - 1$. If no, explain why.

4. (20 points) You're given an undirected, weighted graph G and a minimum spanning tree (MST) T of G. We then decrease the weight of some edge (u, v) in G, where (u, v) is **not** in T.

   Design an algorithm that computes a new MST of G, given the decreased weight of (u, v), in $O(V)$ time. Briefly explain why your algorithm works and why it runs in $O(V)$ time.

   If your algorithm reuses other algorithms, you must reproduce their (potentially modified) pseudocode in full.

   Hint: Since your algorithm must run in $O(V)$ time, you cannot use Prim's or Kruskal's to solve this problem. Consider how you might use T, which is already an MST, to find the new MST.

5. Consider a weighted, directed graph G_1 where all edge weights are positive integers.

   (a) (10 points) Say we created a new graph G_2 by **adding** 2 to all of G_1's edge weights. Would G_1 and G_2 have the same shortest paths? Briefly explain your answer.

   (b) (10 points) Say we created a new graph G_2 by **multiplying** all of G_1's edge weights by 2. Would G_1 and G_2 have the same shortest paths? Briefly explain your answer.

6. (15 points) Design an $O(V(V + E))$ algorithm that solves the all pairs shortest path problem on a weighted graph where all edge weights are 2.

   Briefly explain why your algorithm is correct and runs in time $O(V(V + E))$.

   If your algorithm reuses other algorithms given in the class with some tweaks, you **don't** have to reproduce their pseudocode in full – just describe in detail any changes you'd make.